Modelos de Computación

Memoria Práctica Lex

ANALIZADOR Y DETECTOR DE SPOILERS

Noelia González García
Álvaro Guardia Vera
Andrés Pozo Chiclana

23 de diciembre de 2025

Índice

1. Introducción **2**

2. Desarrollo y funcionamiento **4**
2.1. Máquina de estados y contexto 4
2.2. Detección de patrones 4
2.3. Algoritmo de puntuación 5

3. Instrucciones de compilación y ejecución **6**
3.1. Compilación 6
3.2. Pruebas de ejecución 6

4. Conclusión **8**

1. Introducción

Para el desarrollo de esta práctica hemos implementado un analizador y detector de spoilers para diferentes obras. Hemos desarrollado un programa capaz de procesar reseñas sobre libros, películas o series y calcular automáticamente el grado de spoiler que se ha detectado en el texto introducido.

Nuestro programa no solo detecta palabras sueltas que pueden ser consideradas como spoilers, sino que la detección se divide en varias fases:

- **Detección de patrones generales:** El analizador busca patrones que suelen indicar un spoiler, independientemente de la obra. Estos patrones incluyen revelaciones de finales, muertes de personajes o identidades de asesinos.
- **Detección de patrones específicos:** Además de patrones generales, se incluye una pequeña base de datos con información sobre obras concretas, como *Harry Potter*. De esta forma, si el programa sabe de qué obra se está hablando, buscará y penalizará spoilers específicos de esa obra.
- **Deducción automática de la obra:** En caso de que no se indique el título de la obra, el programa es capaz de deducir el título al leer la reseña. Para ello se ha incluido una base de datos con diferentes personajes y sus respectivas obras. De esta forma, si el analizador detecta los personajes Frodo o Gandalf, sabrá que se está hablando sobre *El Señor de los Anillos*.
- **Cálculo del grado de spoiler:** Cada vez que se detecta un patrón o una palabra peligrosa, se le asigna una puntuación según su gravedad. Al final del análisis, dependiendo de la puntuación final, se etiqueta la reseña como: `SIN SPOILERS`, `LEVE`, `MEDIO` o `GRAVE`.

Además, de las fases anteriores para detección y cálculo de spoilers, el programa incluye:

- **Gestión de STOPWORDS:** El programa uncluye una serie de palabras llamadas STOPWORDS que no afectan al análisis, evitando que palabras comunes como preposiciones o artículos influyan en el resultado.
- **Gestión de spoiler explícito:** El programa detecta si el usuario ha incluido un aviso de spoiler explícito en la reseña, marcando la reseña automáticamente.

Finalmente, para facilitar su uso, el programa puede ser utilizado de dos formas:

- **Modo automático (Cabecera):** Mediante redirección de entrada el sistema detecta y procesa el archivo. El analizador lee la primera línea del archivo buscando el tipo de obra (`TIPO: PELICULA`, `LIBRO` o `SERIE`). El título es opcional; si no se proporciona, el programa pasará a analizar la reseña e intentará adivinarlo según el contexto.

- **Modo interactivo (Menú):** El usuario debe introducir manualmente los datos por terminal para configurar el analizador. Se le pedirá al usuario que especifique el tipo y, opcionalmente, el título, además de la reseña a analizar.

Por último, para facilitar la compilación y ejecución del proyecto, se incluye un `Makefile` y un conjunto de tests en Python para probar el funcionamiento de nuestro programa.

Observación: Todas las rutas de los archivos utilizados son relativas y se suponen correctas siempre que se trabaje desde la carpeta raíz del proyecto.

2. Desarrollo y funcionamiento

Nuestro proyecto se basa en la implementación de un analizador léxico mediante el uso de Flex. Este analizador se ha complementado con código de C++ para la gestión lógica, el cálculo de niveles de spoiler y otras tareas que son más eficientes de realizar en C++ que en Lex.

2.1. Máquina de estados y contexto

Para el análisis se ha diseñado el sistema como un autómata finito de estados. El sistema cambia su comportamiento según el estado actual de la reseña:

- **INITIAL**: Es el estado inicial por defecto de Lex. El analizador espera encontrar una línea del tipo `TIPO: LIBRO/PELICULA/SERIE`. Su función es identificar el tipo de obra sobre la que se va a hablar.
  - Si se detecta el tipo, se pasa al estado `CABECERA`.
  - En caso de no encontrar un tipo válido, o si el archivo empieza directamente con la reseña, el analizador saltará directamente al estado `CUERPO` para analizar el texto.
- **CABECERA**: Este estado tiene como función detectar y procesar el título de la obra.
  - Si se encuentra, se asigna la obra y se pasa al siguiente estado.
  - Si no se especifica el título, saltará al siguiente estado sin procesar el título, que será procesado más tarde en la lógica de deducción.
- **CUERPO**: Es el estado final, tiene como función detectar los patrones y palabras de spoiler. Este estado utiliza tanto las expresiones regulares generales para cualquier obra como las expresiones específicas de nuestras bases de datos.

2.2. Detección de patrones

La detección de spoilers no se basa únicamente en palabras sueltas, sino que se ha implementado un sistema híbrido que combina expresiones regulares y bases de datos específicas.

- **Patrones generales:** Son expresiones regulares de Lex que permiten detectar patrones que indican una revelación de la trama, por ejemplo, muertes o revelaciones de finales.
- **Patrones específicos:** Son reglas específicas para obras y sagas concretas. El analizador combina la aparición de patrones generales con información de las bases de datos para una mayor penalización.

- **Deducción de la obra:** Como es posible que no siempre tengamos el título o que el título no se encuentre en nuestra base de datos específica, se ha incluido una base de datos de personajes. Si el analizador encuentra nombres propios conocidos pero no tiene título, realiza un conteo de coincidencias. La obra con más coincidencias de personajes se asigna automáticamente a la reseña.

2.3. Algoritmo de puntuación

Para calcular el riesgo se ha implementado un sistema de pesos acumulativo en el que cada patrón tiene un peso asociado que determina su gravedad. Para mejorar el sistema de puntuación se han implementado dos sistemas de ventanas deslizantes:

- **Ventana peligrosa:** Al detectar ciertas palabras marcadas como peligrosas, como la frase *"Al final..."*, se activa un multiplicador temporal. En nuestro caso hemos ajustado el sistema para que las siguientes 8 palabras tengan el doble de peso, asumiendo que la siguiente información es un gran spoiler.
- **Ventana de negación:** Esta ventana está diseñada para evitar "falsos positivos". Si se detecta un patrón de negación, como *"No muere..."*, el sistema reduce el peso de las siguientes palabras. Hemos ajustado esta ventana para que las siguientes 8 palabras tengan un peso nulo, así evitamos marcar como spoiler frases que los desmienten.

Al finalizar el análisis, el programa normaliza la puntuación total y asigna un grado de spoiler según unos umbrales definidos: `SIN SPOILER`, `LEVE`, `MEDIO` o `GRAVE`.

3. Instrucciones de compilación y ejecución

Para mejorar y facilitar la compilación del proyecto se ha incluido un `Makefile` que gestiona la compilación automática del mismo. Además, se han incluido una serie de tests en `Python` que nos permiten probar todas las características de nuestro analizador:

1. Detección de patrones.
2. Deducción de obras.
3. Cálculo de gravedad.

3.1. Compilación

Para facilitar la compilación completa del proyecto se incluye un archivo `Makefile` que nos permite automatizar este proceso. Para poder compilar el proyecto correctamente es necesario ejecutar las órdenes desde el directorio raíz del proyecto.

- **Compilación:** Para compilar el proyecto basta con ejecutar el comando `make`. De esta forma se generará el ejecutable de nuestro proyecto listo para ser utilizado.
- **Limpieza:** Si queremos eliminar todos los archivos generados a partir del paso anterior, basta con ejecutar la orden `make clean` y podremos volver a compilar de forma limpia sin restos de la compilación anterior.

3.2. Pruebas de ejecución

Al igual que para la compilación, se ha definido un script de `Python` para automatizar la ejecución de tests de prueba. De la misma forma, es necesario estar en la carpeta raíz del proyecto para poder ejecutar los tests con la orden `python3 test.py`.

Como ya habíamos comentado, nuestro sistema tiene dos modos de funcionamiento:

1. **Por cabecera**
2. **Interactivo**

La idea de estos tests es comprobar que ambos modos y el resto de procesos funcionan correctamente y sin errores. Algunas de las características de nuestro analizador que se prueban son:

- Modo cabecera por redirección de entrada.
- Modo interactivo por menú.
- Comprobación de que las `STOPWORDS` no afectan al análisis.

- Comprobación de cómo afecta la ventana peligrosa y la ventana de negación al análisis.
- Evaluación de cómo afecta la acumulación de pesos específicos frente a únicamente pesos generales.
- Deducción de la obra a partir del contexto de la reseña.
- Comprobación de spoiler explícito en la reseña.
- Prueba de tipo de obra desconocido.

4. Conclusión

El uso combinado de Lex y de C++ nos ha permitido desarrollar un analizador rápido a partir de expresiones regulares y lógicas de Lex y C++, enfocando cada proceso del analizador según la herramienta que mejor lo resuelve:

- La lógica de Lex nos ha permitido realizar un escaneo rápido de las expresiones definidas y asociarles un funcionamiento mediante el reconocimiento de patrones.
- La lógica de C++ nos ha facilitado el desarrollo de características del analizador que en Lex serían mucho más complejas de implementar, como los sistemas de ventanas deslizantes o la deducción de obras mediante el conteo de personajes.

Sin embargo, aunque con este sistema hemos obtenido buenos resultados, siguen existiendo errores y posibles mejoras que son difíciles de corregir únicamente con la lógica actual. Algunos puntos que podrían mejorar, ya sea mediante una lógica más potente o ampliando la base de conocimiento, son:

- **Análisis sintáctico:** La detección de ironías o frases con estructuras gramaticales complejas podría corregirse integrando un analizador sintáctico que permitan analizar de forma más profunda las frases.
- **Gestión de datos:** La capacidad de detección específica y de deducción automática podría mejorarse simplemente ampliando las bases de datos actuales. Otra opción sería conectar el analizador a APIs o bases de datos externas para actualizar automáticamente las obras y personajes.
- **Refinamiento de la negación:** Aunque la ventana de negación reduce los falsos positivos, podrían reducirse aún más incorporando más patrones que activen la ventana de negación.
- **Flexibilidad en la cabecera:** Actualmente el sistema supone un entorno controlado donde `TIPO` precede a `TITULO`. Una mejora lógica permitiría aceptar el orden inverso para evitar errores de formato del usuario.
- **Deducción total:** Otra mejora sería incluir una base de datos de personajes inversa que permita deducir el `TIPO` de obra (Libro, Serie, Película) si el usuario lo omite, basándose en la obra previamente identificada por el contexto.